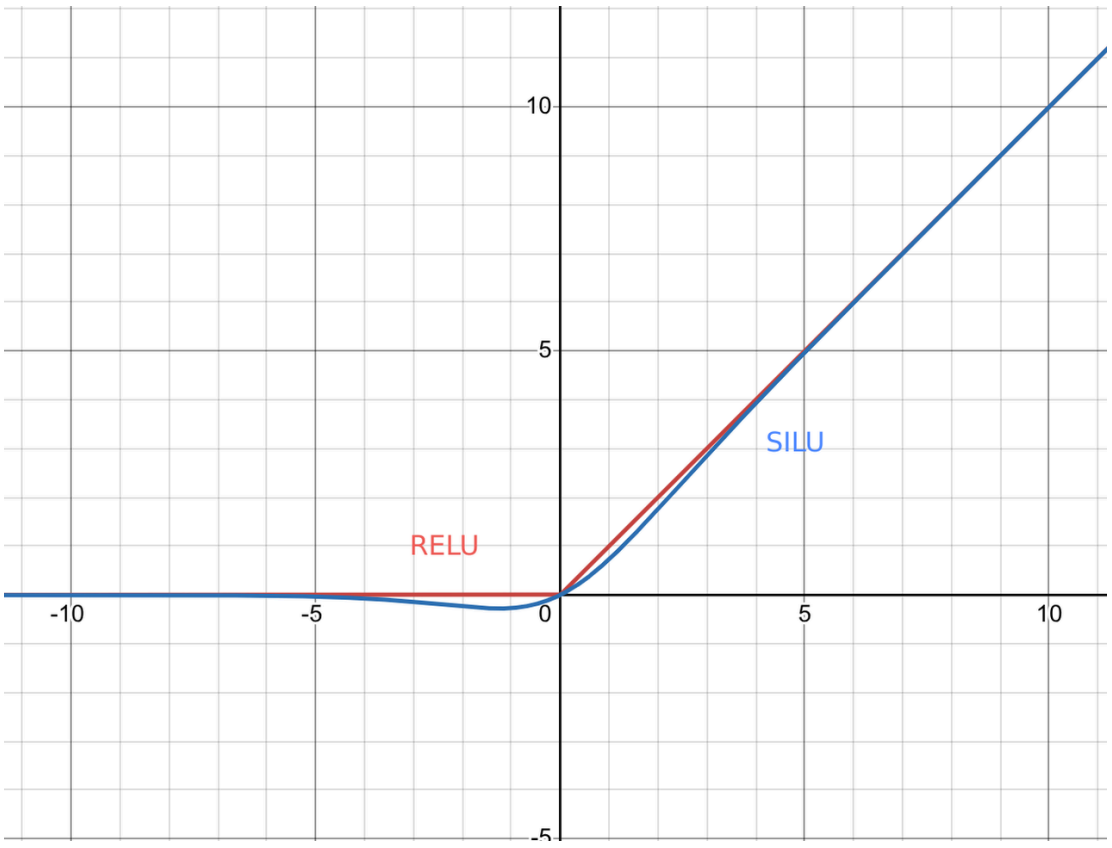
Stochastic activations

> #Activation #Pretrain

- What problem does this framework solve?
  - A popular activation function, SILU, is used in the FFN of transformers nowadays. This function will provide non-zero values for both $x \leq 0$ and $x > 0$, and the resultant output of the model with this activation function is high-quality. **However, due to the low-sparsity of SILU, the computes needed are high.**
  - RELU, on the other hand, is the total opposite: it is sparse and light-weighted. However, as $RELU(x) = 0 \; \forall \; x \leq 0$, **there will be a lot of dead neurons**.
  - Is there a way to combine the advantages of these two?
- What does this framework do?
  - The paper proposes two strategies: **Swi+FT** and **Stocha**.
  - **Swi+FT**
    - Define α as the percentage of the total steps. We use the last α percentage of steps to use RELU as the activation function of the FFN during training.
    - The 1st $1 - \alpha$ of the stage uses SILU.
    - With SILU most of the time, the dead neuron problem is avoided and the model will produce a high precision response, and during the final period of training, use RELU to make the model "get used to" the RELU activation function, which is commonly used during inference.
    - These two stages can naturally connected as they have similar shapes:



- **Stocha**
  - Let ω be a random binary variable where $\mathbb{P}(\omega = 1) = p,\ \mathbb{P}(\omega = 0) = 1 - p$.
  - Let the **stocha activation function** be:

    $$\Psi_p(x) = (1 - \omega) \times RELU(x) + \omega \times SILU(x) \text{ if } x < 0 \text{ otherwise } SILU(x)$$

    - Meaning the activation function has p be $SILU$ and $1 - p$ be $RELU$.
- It is observed that using Swi+FT only will not achieve the best effect, so the paper proposes to use Stocha together with Swi+FT:

  $$\begin{aligned} &\text{for the 1st } 1 - \alpha \text{ percentage of steps :} \\ &\qquad\qquad Stocha(x) \\ &\text{for the last } \alpha \text{ percentage of steps :} \\ &\qquad\qquad RELU(x) \end{aligned}$$

  - The time scheduling here uses Swi+FT but replaces the 1st $1 - \alpha$ percentage of steps with Stocha.

- Experiment Results
  - Inference Acceleration and High Sparsity: Models trained using Stocha combined with Swi+FT achieve up to approximately 90% activation sparsity during inference by exclusively employing RELU. This directly translates to approximately 1.65x acceleration on CPUs and approximately 1.5x acceleration for FFN layers on GPUs.
  - Performance Rivals Top-Tier Models: The RELU model trained with the hybrid strategy significantly outperforms pure RELU models across multiple benchmarks, including code generation and common-sense reasoning, while nearly matching the baseline model using SILU.
  - Unexpected bonus (generative diversity): Retaining Stocha's randomness during inference (without switching to pure RELU) serves as an alternative to traditional “temperature sampling,” generating text sequences with richer vocabulary and greater diversity.
- Work Cited
  - Paper
  - Chat History
  - NotebookLM